

Reviewer Pack — Minimal Index of Algebraic Coadjointness and Resolution Proo...

Entry 79e17b977914 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 14:25:49 UTC

Minimal Index of Algebraic Coadjointness and Resolution Proof Width

Entry ID: 79e17b977914 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 14:25:49 UTC

1. Conjecture

Field A (mathematical branch): Lie Theory (Coadjoint Representations) **Field B** (complexity object): Resolution Proof Complexity

Statement:

For any given Conjunctive Normal Form (CNF) φ with n variables, the resolution proof width of φ is bounded by the minimal index of algebraic coadjointness among all linear operators on the vector space associated with φ .

Rationale (proposer's reasoning):

Algebraic coadjointness can capture the noncommutative nature of certain mathematical structures, and its minimal index might reveal insights into the complexity of resolution proofs. The relationship between this algebraic invariant and proof width could potentially expose new structural properties in computational complexity.

Taxonomy category: Resolution Proof Complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 18381ccc572cf5a4

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all CNFs φ with n variables ($n \leq 40$), the correlation coefficient between the minimal index of algebraic coadjointness and the resolution proof width is ≥ 0.7 over 30 random seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	0.95	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal index of algebraic coadjointness" AND "resolution proof complexity" - "Lie theory" AND "algebraic coadjointness" AND CNF" - "Conjunctive Normal Form" AND resolution AND "index of algebraic coadjointness"

Top relevant hits considered: - [s2:d52e959a7093edf79a9b43577cc9a2c249117513] Algebraic K-theory of discrete subgroups of Lie groups - [s2:10.48175/ijarsct-31208] Group Theory Used in Computer Graphics: Innovative Topics & Applications

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(2 * n):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
            if all(clause[i] != -clause[j] for j in range(i)):
                clauses.append(clause)
        return clauses

    def vector_space_from_cnf(cnf):
        variables = set(abs(lit) for clause in cnf for lit in clause)
        n_vars = max(variables)
        V = [[0] * (n_vars + 1) for _ in range(n_vars + 1)]
        for clause in cnf:
            for i, lit in enumerate(clause):
                if lit > 0:
                    V[lit][i + 1] += 1
                else:
                    V[-lit][i + 1] -= 1
        return V

    def minimal_index_of_adjointness(V):
        n = len(V)
        I = [[0] * n for _ in range(n)]
```

```

    for i in range(n):
        for j in range(n):
            if i != j:
                I[i][j] = sum(V[i][k] * V[j][k] for k in range(1, n + 1))
    min_index = float('inf')
    for row in I:
        min_index = min(min_index, max(row) - min(row))
    return min_index

def resolution_width(cnf):
    stack = []
    visited = set()
    while cnf:
        unit_clause = next((c for c in cnf if len(c) == 1), None)
        if not unit_clause:
            break
        lit = unit_clause[0]
        visited.add(abs(lit))
        cnf = [c for c in cnf if lit not in c and -lit not in c]
        stack.append(unit_clause)
    return max(len(stack), len(visited))

n_values = [5, 10, 15, 20, 30, 40]
metrics = []

for n in n_values:
    cnf = generate_cnf(n)
    V = vector_space_from_cnf(cnf)
    min_index = minimal_index_of_adjointness(V)
    width = resolution_width(cnf)
    metrics.append((min_index, width))

if not metrics:
    return {
        "metric_name": "resolution_width",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

min_indices, widths = zip(*metrics)
correlation_coefficient = sum((mi - mean(min_indices)) * (w - mean(widths)) for mi, w in metrics)

return {
    "metric_name": "resolution_width",
    "metric_value": correlation_coefficient,
    "instances_tested": len(metrics),
    "n_max": max(n_values),
    "conjecture_holds": correlation_coefficient >= 0.7,
    "counterexample": ""
}

def mean(values):
    return sum(values) / len(values)

```

```

def std_dev(values):
    avg = mean(values)
    return math.sqrt(sum((x - avg) ** 2 for x in values) / len(values))

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 1000003) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = mean([r["metric_value"] for r in results if r["metric_value"] is not None])
    std_value = std_dev([r["metric_value"] for r in results if r["metric_value"] is not None])
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unmet_acceptance_criterion")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_817b20aa.py", line 111, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_817b20aa.py", line 70, in run_trial
    cnf = generate_cnf(n)
         ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_817b20aa.py", line 25, in generate_cnf
    if all(clause[i] != -clause[j] for j in range(i)):
           ^

```

NameError: name 'i' is not defined. Did you mean: 'id'?

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed due to an undefined variable 'i', which prevented the production of data necessary to evaluate the conjecture. | next: Debug and correct the error in the test code, then rerun the test with a valid CNF generation process.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11951
2	preregistration	ollama_remote	glm4:latest	0	0	9093
3	novelty	ollama_remote	glm4:latest	0	0	8232
4	novelty	ollama_remote	glm4:latest	0	0	9743
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18540
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15435
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10960
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12392
9	judge	ollama_remote	glm4:latest	0	0	16283

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 112629 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/79e17b977914.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/79e17b977914.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/79e17b977914.tar.gz` (if generated)